K.R. MANGALAM UNIVERSITY

Database Management Lab File
(ENCS205)

Bachelor of Technology
in
Computer Science and Engineering

Submitted by
Raghav Arora (2401010114)

B. Tech CSE (Core) – 4 Semester

Submitted To:
MS. Mansi Kajal

School of Engineering & Technology

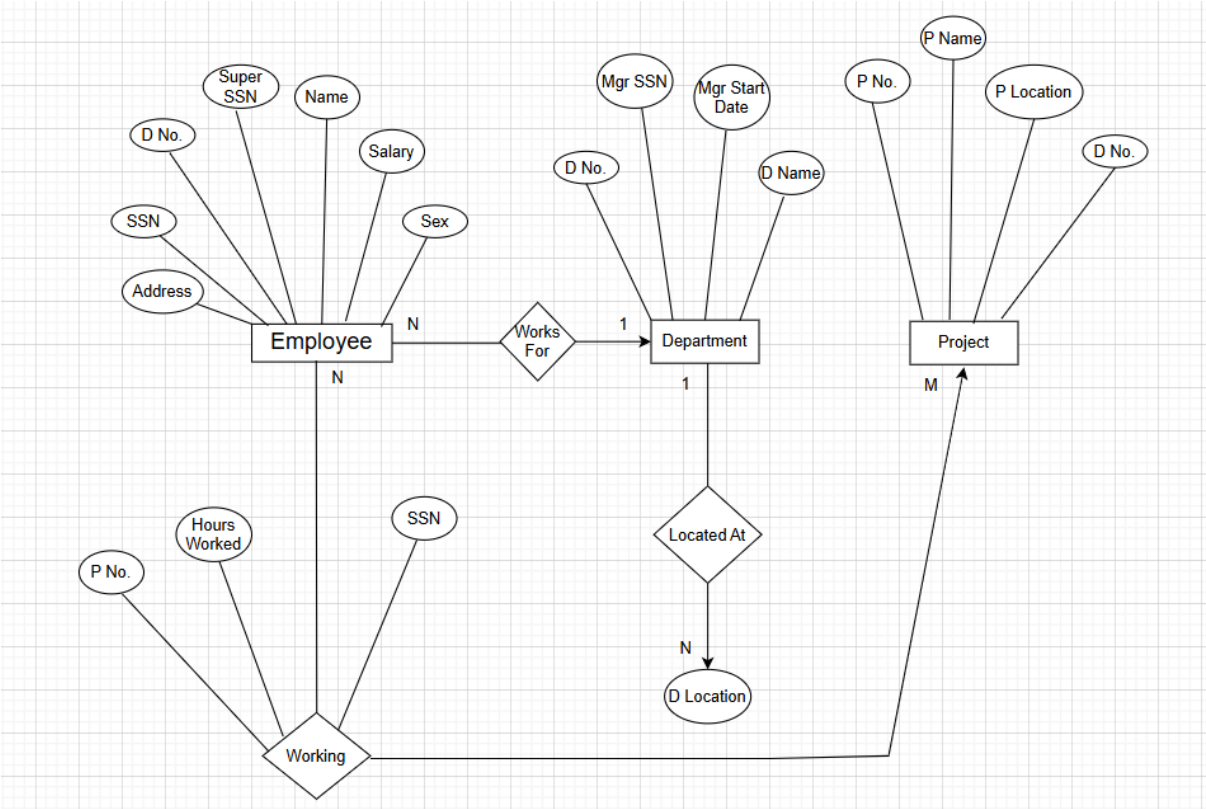
K. R. MANGALAM UNIVERSITY Sohna, Haryana
122103, India Jan-2026

Experiment 1

1. **Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:**

COMPANY DATABASE:

EMPLOYEE (SSN, Name, Address, Sex, Salary, Super SSN, D No.) DEPARTMENT (D No., D Name, Mgr SSN, Mgr Start Date) DLOCATION (D Location)
PROJECT (P No., P Name, P Location, D No.)
WORKING (SSN, P No., Hours Worked)



Explanation of ER Diagram – Company Database :

- This ER diagram represents a company database that stores information about employees, departments, locations, and projects.

- The Employee entity stores details such as SSN, name, address, sex, salary, supervisor SSN, and department number. Each employee works for a department. This relationship is shown using WORKS FOR. Many employees can work for one department, but each employee works for only one department.
- The Department entity stores department information such as

department number, department name, manager SSN, and manager start date. A department can have multiple locations. This is shown using the LOCATED AT relationship between Department and D Location.

- The Project entity stores information about projects such as project number, project name, project location, and department number. Each project is controlled by one department, and one department can control many projects.

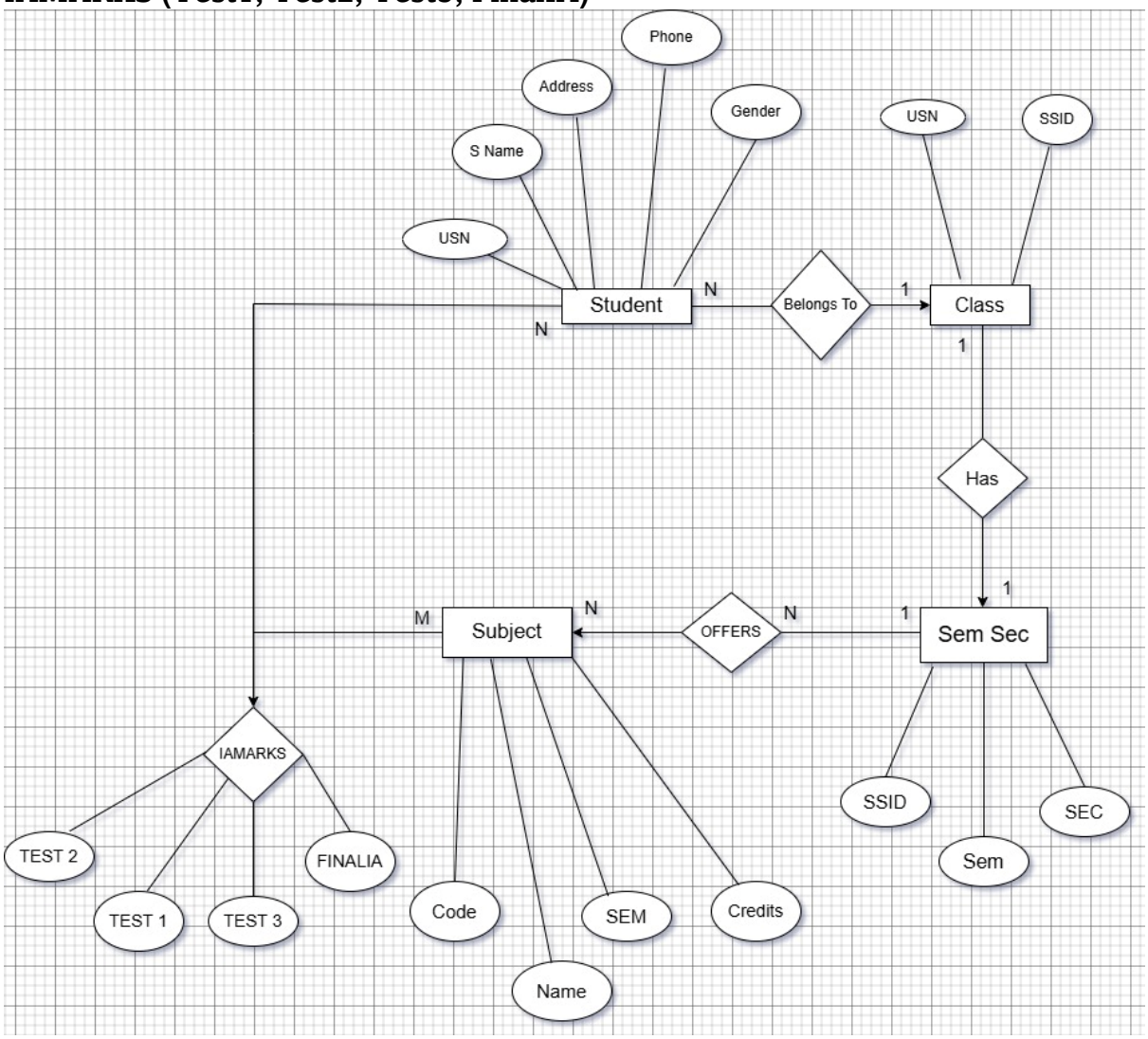
- Employees work on different projects. This many-to-many relationship is shown using the WORKS ON relationship. The number of hours an employee works on a project is stored as an attribute of this relationship.

EXPERIMENT-2

2. Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

COLLEGE DATABASE:

STUDENT (USN, S Name, Address, Phone, Gender) SEMSEC (SSID, Sem, SEC)
CLASS (USN, SSID)
SUBJECT (Code, Name, Sem, Credits)
IAMARKS (Test1, Test2, Test3, FinalIA)



Explanation of ER Diagram – College Database :

- This ER diagram represents a college database that stores information about students, classes, semester–sections, subjects, and internal

assessment marks.

- In this diagram, the Student entity stores the basic details of a student such as USN, student name, address, phone number, and gender. Every student belongs to a particular class. This relationship is shown using the BELONGS TO relationship. Many students can belong to the same class, but a student can belong to only one class.

- The Class entity represents a group of students. Each class is associated with exactly one semester and section, which is represented by the SemSec entity. The relationship between Class and SemSec is one-to-one, meaning each class has only one semester-section and each semester-section belongs to one class.

- The SemSec entity stores information about the semester and section. A semester-section offers multiple subjects. This is shown by the OFFERS relationship between SemSec and Subject. One semester-section can offer many subjects, but each subject is offered in one specific semester-section.

- The Subject entity stores subject details such as subject code, subject name, semester, and credits. Students study multiple subjects, and each subject is studied by many students. This many-to-many relationship is represented using the IAMARKS relationship.

- The IAMARKS relationship stores the internal assessment marks of students for each subject. It includes Test1, Test2, Test3, and Final IA. This relationship connects Student and Subject and stores marks for every student–subject combination.

Experiment 3

Database Schema for a customer-sale scenario
Customer(Cust id : integer, cust_name: string)
Item(item_id: integer, item_name: string, price: integer)
Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)
For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.

```
CREATE DATABASE IF NOT EXISTS CustomerSaleDB;
USE CustomerSaleDB;

CREATE TABLE Customer (
   cust_id INT PRIMARY KEY,
   cust_name VARCHAR(100) NOT NULL
);


CREATE TABLE Item (
   item_id INT PRIMARY KEY,
   item_name VARCHAR(100) NOT NULL,
   price INT NOT NULL CHECK (price > 0)
);

CREATE TABLE Sale (
   bill_no INT,
   bill_date DATE NOT NULL,
   cust_id INT,
   item_id INT,
   qty_sold INT NOT NULL CHECK (qty_sold > 0),
   PRIMARY KEY (bill_no, item_id),
   FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
   FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

	cust_id	cust_name
*	NULL	NULL

	item_id	item_name	price
*	NULL	NULL	NULL

	bill_no	bill_date	cust_id	item_id	qty_sold
*	NULL	NULL	NULL	NULL	NULL

2. Insert around 10 records in each of the tables.

bill_no	cust_name	item_id
1001	Ishaan	101
1002	Mahin	102
1003	Aditya	104
1004	Ayush	105
1005	Priya	108
1006	Karan	110
1007	Neha	103
1008	Rohit	109
1009	Simran	106
1010	Vikas	107

```
INSERT INTO Customer VALUES
(1,'Ishaan'),
(2,'Mahin'),
(3,'Aditya'),
(4,'Ayush'),
(5,'Priya'),
(6,'Karan'),
(7,'Neha'),
(8,'Rohit'),
(9,'Simran'),
(10,'Vikas');

INSERT INTO Item VALUES
(101,'Keyboard',50000),
(102,'Mouse',500),
(103,'Keyboard',800),
(104,'Monitor',12000),
(105,'USB Cable',200),
(106,'Headphones',1500),
(107,'Charger',900),
(108,'Tablet',25000),
(109,'Smartphone',30000),
(110,'Pen Drive',700);

INSERT INTO Sale VALUES
(1001,CURDATE(),1,101,1),
(1002,CURDATE(),2,102,2),
(1003,CURDATE(),3,104,1),
(1004,CURDATE(),4,105,3),
(1005,CURDATE(),5,108,1),
(1006,CURDATE(),6,110,4),
(1007,CURDATE(),7,103,2),
(1008,CURDATE(),8,109,1),
(1009,CURDATE(),9,106,1),
(1010,CURDATE(),10,107,2);
```

cust_id	cust_name
1	Ishaan
2	Mahin
3	Aditya
4	Ayush
5	Priya
6	Karan
7	Neha
8	Rohit
9	Simran
10	Vikas
NULL	NULL

item_id	item_name	price
101	Keyboard	50000
102	Mouse	500
103	Keyboard	800
104	Monitor	12000
105	USB Cable	200
106	Headphones	1500
107	Charger	900
108	Tablet	25000
109	Smartphone	30000
110	Pen Drive	700
NULL	NULL	NULL

3. List all the bills for the current date with the customer names and item numbers.

```
SELECT s.bill_no, c.cust_name, s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();
```

bill_no	bill_date	cust_id	item_id	qty_sold
1001	2026-02-28	1	101	1
1002	2026-02-28	2	102	2
1003	2026-02-28	3	104	1
1004	2026-02-28	4	105	3
1005	2026-02-28	5	108	1
1006	2026-02-28	6	110	4
1007	2026-02-28	7	103	2
1008	2026-02-28	8	109	1
1009	2026-02-28	9	106	1
1010	2026-02-28	10	107	2
NULL	NULL	NULL	NULL	NULL

4. List the total Bill details with the quantity sold, price of the item and the final amount.

```
SELECT
   s.bill_no,
   s.qty_sold,
   i.price,
   (s.qty_sold * i.price) AS final_amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

bill_no	qty_sold	price	final_amount
1001	1	50000	50000
1002	2	500	1000
1003	1	12000	12000
1004	3	200	600
1005	1	25000	25000
1006	4	700	2800
1007	2	800	1600
1008	1	30000	30000
1009	1	1500	1500
1010	2	900	1800

5. List the details of the customer who have bought a product which has a price>200.

```
SELECT DISTINCT c.cust_id, c.cust_name
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;
```

cust_id	cust_name
1	Ishaan
2	Mahin
7	Neha
3	Aditya
9	Simran
10	Vikas
5	Priya
8	Rohit
6	Karan

6. Give a count of how many products have been bought by each customer

```
SELECT
   c.cust_id,
   c.cust_name,
   SUM(s.qty_sold) AS total_products_bought
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
GROUP BY c.cust_id, c.cust_name;
```

	cust_id	cust_name	total_products_bought
▶	1	Ishaan	1
	2	Mahin	2
	3	Aditya	1
	4	Ayush	3
	5	Priya	1
	6	Karan	4
	7	Neha	2
	8	Rohit	1
	9	Simran	1
	10	Vikas	2

7. Give a list of products bought by a customer having cust_id as 5.

```
SELECT i.item_id, i.item_name
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;
```

	item_id	item_name
▶	108	Tablet

8. List the item details which are sold as of today.

```
SELECT DISTINCT i.*
FROM Item i
JOIN Sale s ON i.item_id = s.item_id
WHERE s.bill_date = CURDATE();
```

	item_id	item_name	price
▶	101	Keyboard	50000
	102	Mouse	500
	104	Monitor	12000
	105	USB Cable	200
	108	Tablet	25000
	110	Pen Drive	700
	103	Keyboard	800
	109	Smartphone	30000
	106	Headphones	1500
	107	Charger	900

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

```
CREATE OR REPLACE VIEW Bill_View AS
SELECT
    s.bill_no,
    s.bill_date,
```

```
    s.cust_id,
    s.item_id,
    i.price,
    s.qty_sold,
    (s.qty_sold * i.price) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
SELECT * FROM Bill_View;
```

bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
1001	2026-02-28	1	101	50000	1	50000
1002	2026-02-28	2	102	500	2	1000
1003	2026-02-28	3	104	12000	1	12000
1004	2026-02-28	4	105	200	3	600
1005	2026-02-28	5	108	25000	1	25000
1006	2026-02-28	6	110	700	4	2800
1007	2026-02-28	7	103	800	2	1600
1008	2026-02-28	8	109	30000	1	30000
1009	2026-02-28	9	106	1500	1	1500
1010	2026-02-28	10	107	900	2	1800

10. Create a view which lists the daily sales date wise for the last one week

```
CREATE OR REPLACE VIEW Weekly_Sales AS
SELECT
    s.bill_date,
    SUM(s.qty_sold * i.price) AS total_sales
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY s.bill_date;
SELECT * FROM Weekly_Sales;
```

Bill_date:	2026-02-28
Total_sales:	126300

EXPERIMENT 4

TCL Commands – COMMIT, SAVEPOINT, ROLLBACK

Objective:

Consider the following table: CLASS (ID, Name) with initial records: 1-Bravo, 2-Alex, 4-Cheng. Demonstrate the use of TCL commands: INSERT, COMMIT, UPDATE, SAVEPOINT, ROLLBACK.

4.1 – CLASS Table TCL Script

Code:

```
SET autocommit = 0;

CREATE TABLE class (
    id INT,
    name VARCHAR(50)
);

INSERT INTO class VALUES (1,'Bravo');
INSERT INTO class VALUES (2,'Alex');
INSERT INTO class VALUES (4,'Cheng');
INSERT INTO class VALUES (5,'Rahul');
COMMIT;

UPDATE class SET name='Abhijeet' WHERE id='5';
SAVEPOINT A;

INSERT INTO class VALUES (6,'Chris');
SAVEPOINT B;

INSERT INTO class VALUES (7,'Bravo');
SAVEPOINT C;

SELECT * FROM class;
ROLLBACK TO B;
SELECT * FROM class;
ROLLBACK TO A;
```

Explanation:

1. autocommit is disabled so all changes are part of a transaction.
2. Initial 4 rows inserted and COMMITted (permanently saved).
3. Row 5 updated to 'Abhijeet', SAVEPOINT A marked.
4. Row 6 ('Chris') inserted, SAVEPOINT B marked.
5. Row 7 ('Bravo') inserted, SAVEPOINT C marked.
6. After first SELECT: rows 1,2,4,5(Abhijeet),6,7 visible.
7. ROLLBACK TO B: row 7 removed. Rows 1,2,4,5,6 remain.
8. ROLLBACK TO A: row 6 also removed. Rows 1,2,4,5(Abhijeet) remain.

EXPERIMENT 4.2

TCL Commands – ACCOUNT_TXN Table

Initial Table: ACCOUNT_TXN

Acc_ID | Holder_Name | Balance
101 | Aman | 5000
102 | Deepak | 7000
103 | Chirag | 6000

Code:

```
SET autocommit = 0;
DROP TABLE IF EXISTS ACCOUNT_TXN;

CREATE TABLE ACCOUNT_TXN (
    Acc_ID INT PRIMARY KEY,
    Holder_Name VARCHAR(50),
    Balance INT
);

INSERT INTO ACCOUNT_TXN VALUES
(101,'Aman',5000), (102,'Deepak',7000), (103,'Chirag',6000);

START TRANSACTION;
INSERT INTO ACCOUNT_TXN VALUES (104,'Diya',4000);
SAVEPOINT S1;

UPDATE ACCOUNT_TXN SET Balance = Balance + 1500 WHERE Acc_ID = 101;
UPDATE ACCOUNT_TXN SET Balance = Balance - 1000 WHERE Acc_ID = 102;
SAVEPOINT S2;

DELETE FROM ACCOUNT_TXN WHERE Acc_ID = 103;
INSERT INTO ACCOUNT_TXN VALUES (105,'Eshan',3000);
SAVEPOINT S3;

UPDATE ACCOUNT_TXN SET Holder_Name = 'Bhavna Sharma' WHERE Acc_ID = 102;
SELECT * FROM ACCOUNT_TXN;

ROLLBACK TO S2;
SELECT * FROM ACCOUNT_TXN;

INSERT INTO ACCOUNT_TXN VALUES (106,'Farah',4500);
SAVEPOINT S4;

UPDATE ACCOUNT_TXN SET Balance = Balance + 500 WHERE Acc_ID IN (101,104);
SELECT * FROM ACCOUNT_TXN;

ROLLBACK TO S4;
SELECT * FROM ACCOUNT_TXN;
COMMIT;
```

Final Output (after COMMIT):

101 | Aman | 6500
102 | Deepak | 6000
103 | Chirag | 6000
104 | Diya | 4000
106 | Farah | 4500

EXPERIMENT 5

SQL Queries on EMPLOYEES Table

Table Schema:

EMPLOYEES (Employee_Id, First_Name, Last_Name, Email, Phone_Number, Hire_Date, Job_Id, Salary, Commission_Act, Manager_Id, Department_Id)

Table Creation & Data Insertion:

```
CREATE database emp;
USE emp;

CREATE TABLE EMPLOYEES (
    Employee_Id INT PRIMARY KEY,
    First_Name VARCHAR(50),
    Last_Name VARCHAR(50),
    Email VARCHAR(100),
    Phone_Number VARCHAR(20),
    Hire_Date DATE,
    Job_Id VARCHAR(20),
    Salary INT,
    Commission_Act DECIMAL(5,2),
    Manager_Id INT,
    Department_Id INT
);

INSERT INTO EMPLOYEES VALUES
(101,'John','Smith','john.smith@email.com','9876543210','2020-01-
15','IT_PROG',6000,NULL,100,60),
(102,'Sara','Connor','sara.connor@email.com','9876543211','2019-03-
10','HR_REP',4500,NULL,101,70),
(103,'Mike','Austin','mike.austin@email.com','9876543212','2021-06-
20','SA_REP',5000,0.10,100,80),
... (15 records total) ...;
```

(a) Find employee id, names, salaries of all employees:

```
SELECT Employee_Id, First_Name, Last_Name, Salary
FROM EMPLOYEES;
```

Output: Lists all 15 employees with their ID, first name, last name, and salary.

(b) List employees who work under Manager 100:

```
SELECT *
FROM EMPLOYEES
WHERE Manager_Id = 100;
```

Output: Returns employees 101 (John), 103 (Mike), 107 (Noah), 110 (Sophia), 113 (Lucas), 115 (William) — all reporting to manager 100.

(c) Find employees with salary >= 4800:

```
SELECT First_Name, Last_Name
FROM EMPLOYEES
```

```
WHERE Salary >= 4800;
```
Output: John Smith, Mike Austin, Emma Watson, Liam Brown, Noah Wilson, Ethan Anderson, Sophia Thomas, James Jackson, Isabella White, Mia Martin, William Austin.

(d) List employees whose last name is 'AUSTIN':

```
SELECT *
FROM EMPLOYEES
WHERE Last_Name = 'AUSTIN';
```
Output: Mike Austin (103, SA_REP, Salary 5000) and William Austin (115, IT_PROG, Salary 7500).

(e) Find employees in departments 60, 70, and 80:

```
SELECT First_Name, Last_Name
FROM EMPLOYEES
WHERE Department_Id IN (60, 70, 80);
```
Output: All 15 employees (departments 60, 70, 80 cover all records in the dataset).

(f) Display unique Manager_Ids:

```
SELECT DISTINCT Manager_Id
FROM EMPLOYEES;
```
Output: 100, 101, 102 — the three unique managers in the table.

EXPERIMENT 6

Updating Records – Client_master Table

Table Creation:

```
CREATE DATABASE Client_master;
USE Client_master;

CREATE TABLE Client_master (
    ClientNO VARCHAR(10) PRIMARY KEY,
    Name VARCHAR(50),
    Address VARCHAR(100),
    City VARCHAR(50),
    State VARCHAR(50),
    bal_due DECIMAL(10,2)
);
```

(a) Insert five records:

```
INSERT INTO Client_master VALUES
('C106','Anjali Kapoor','Sector 18','Noida','Uttar Pradesh',4800),
('C107','Vikas Malhotra','Connaught Place','Delhi','Delhi',8000),
('C108','Sneha Reddy','Banjara Hills','Hyderabad','Telangana',6200),
('C109','Arjun Nair','Marine Drive','Mumbai','Maharashtra',5300),
('C110','Karan Patel','SG Highway','Ahmedabad','Gujarat',4500);
```

(b) Find clients with bal_due > 5000:

```
SELECT Name
FROM Client_master
WHERE bal_due > 5000;
```

Output: Vikas Malhotra (8000), Sneha Reddy (6200), Arjun Nair (5300).

(c) Change bal_due of ClientNO C123 to Rs. 5100:

```
UPDATE Client_master
SET bal_due = 5100
WHERE ClientNO = 'C123';
```

Note: C123 does not exist in the inserted data, so 0 rows are affected.

(d) Rename Client_master table to Client12:

```
ALTER TABLE Client_master
RENAME TO Client12;
```

The table is now accessible as Client12.

(e) Display bal_due column as BALANCE:

```
SELECT ClientNO, Name, Address, City, State, bal_due AS BALANCE
FROM Client12;
```

Output: All client records with the bal_due column header displayed as BALANCE.

EXPERIMENT 7

ROLLBACK and COMMIT Commands – Teacher Table

Objective:

Create a Teacher table with fields: Name, DeptNo, DateOfJoining, DeptName, Location, Salary. Perform insert, increment, rollback, increment, and commit operations.

Table Creation:

```
CREATE DATABASE IF NOT EXISTS school_management;
USE school_management;
SET SQL_SAFE_UPDATES = 0;

DROP TABLE IF EXISTS Teacher;
CREATE TABLE Teacher (
    TeacherName VARCHAR(50),
    DeptNo INT,
    DateOfJoining DATE,
    DeptName VARCHAR(50),
    LocationName VARCHAR(50),
    SalaryAmt INT
);
```

(a) Insert five records:

```
INSERT INTO Teacher VALUES
('amit kumar', 10, '2021-01-10', 'mathematics', 'delhi', 40000),
('sunita devi', 20, '2020-03-15', 'commerce', 'mumbai', 35000),
('rahul sharma', 10, '2022-05-22', 'mathematics', 'delhi', 42000),
('priya nair', 30, '2019-11-01', 'science', 'kochi', 50000),
('aniket das', 20, '2023-02-14', 'commerce', 'kolkata', 32000);
```

(b) 25% salary increment for Mathematics Department:

```
START TRANSACTION;
UPDATE Teacher
SET SalaryAmt = SalaryAmt * 1.25
WHERE DeptName = 'mathematics';
SELECT * FROM Teacher WHERE DeptName = 'mathematics';
```

Output after update: amit kumar -> 50000, rahul sharma -> 52500.

(c) Perform ROLLBACK:

```
ROLLBACK;
SELECT * FROM Teacher WHERE DeptName = 'mathematics';
```

Output after rollback: Salaries restored to original — amit kumar: 40000, rahul sharma: 42000.

(d) 15% salary increment for Commerce Department:

```
START TRANSACTION;
UPDATE Teacher
SET SalaryAmt = SalaryAmt * 1.15
WHERE DeptName = 'commerce';
```

sunita devi: 35000 -> 40250, aniket das: 32000 -> 36800.

(e) Perform COMMIT:

```
COMMIT;
SELECT * FROM Teacher;
SET SQL_SAFE_UPDATES = 1;
```

Final Table Output:

amit kumar | 10 | 2021-01-10 | mathematics | delhi | 40000

sunita devi | 20 | 2020-03-15 | commerce | mumbai | 40250

rahul sharma | 10 | 2022-05-22 | mathematics | delhi | 42000

priya nair | 30 | 2019-11-01 | science | kochi | 50000

aniket das | 20 | 2023-02-14 | commerce | kolkata | 36800

EXPERIMENT 8

Aggregate Functions & Date Queries – Sales Table

Table Creation:

```
CREATE DATABASE IF NOT EXISTS sales_db;
USE sales_db;
DROP TABLE IF EXISTS Sales;

CREATE TABLE Sales (
    SalesNo INT PRIMARY KEY,
    SalesName VARCHAR(50),
    BranchName VARCHAR(50),
    SalesAmount INT,
    DateOfBirth DATE
);
```

(a) Insert five records:

```
INSERT INTO Sales VALUES
(101, 'arnav gupta', 'north', 45000, '1995-12-21'),
(102, 'sana sheikh', 'south', 32000, '1998-05-15'),
(103, 'kabir malhotra', 'north', 55000, '1996-12-09'),
(104, 'isha patel', 'east', 40000, '1997-12-05'),
(105, 'vihaan nair', 'south', 28000, '1999-08-22');
```

(b) Total salesamount per branch:

```
SELECT BranchName, SUM(SalesAmount) AS TotalSales
FROM Sales
GROUP BY BranchName;
```

Output: north -> 100000, south -> 60000, east -> 40000

(c) Average salesamount per branch:

```
SELECT BranchName, AVG(SalesAmount) AS AverageSales
FROM Sales
GROUP BY BranchName;
```

Output: north -> 50000.0000, south -> 30000.0000, east -> 40000.0000

(d) Salesmen born in December (formatted as dd-Mon-yy):

```
SELECT SalesName, DATE_FORMAT(DateOfBirth, '%d-%b-%y') AS FormattedDOB
FROM Sales
WHERE MONTH(DateOfBirth) = 12;
```

Output:

arnav gupta -> 21-Dec-95

kabir malhotra -> 09-Dec-96

isha patel -> 05-Dec-97

(e) Name and DOB sorted by month name alphabetically:

```
SELECT SalesName, DateOfBirth
FROM Sales
ORDER BY MONTHNAME(DateOfBirth) ASC;
```

Output (alphabetical month order):
vihaan nair -> 1999-08-22 (August)
arnav gupta -> 1995-12-21 (December)
kabir malhotra -> 1996-12-09 (December)
isha patel -> 1997-12-05 (December)
sana sheikh -> 1998-05-15 (May)

EXPERIMENT 9

GRANT, REVOKE and SAVEPOINT – DCL Commands

Schema:

Departments (DeptNo, DeptName, DeptLocation)

Employees (EmpId, EmpName, EmpSalary, DeptNo)

Table Creation:

```
CREATE DATABASE IF NOT EXISTS company_records;
USE company_records;

CREATE TABLE IF NOT EXISTS Departments (
    DeptNo INT PRIMARY KEY,
    DeptName VARCHAR(50),
    DeptLocation VARCHAR(50)
);

CREATE TABLE IF NOT EXISTS Employees (
    EmpId INT PRIMARY KEY,
    EmpName VARCHAR(50),
    EmpSalary INT,
    DeptNo INT,
    FOREIGN KEY (DeptNo) REFERENCES Departments(DeptNo)
);

CREATE USER IF NOT EXISTS 'dept_manager'@'localhost' IDENTIFIED BY
'secure_pass123';
```

(a) Grant ALL privileges on Employees table:

```
GRANT ALL PRIVILEGES ON company_records.Employees TO
'dept_manager'@'localhost';
```

This grants the user dept_manager full access (SELECT, INSERT, UPDATE, DELETE, etc.) on the Employees table.

(b) Grant SOME privileges on Employees table:

```
GRANT SELECT, UPDATE ON company_records.Employees TO
'dept_manager'@'localhost';
```

This grants only SELECT and UPDATE permissions — the user cannot INSERT or DELETE.

(c) Revoke ALL privileges from Employees table:

```
REVOKE ALL PRIVILEGES ON company_records.Employees FROM
'dept_manager'@'localhost';
```

All previously granted privileges are removed from the user.

(d) Revoke SOME privileges from Employees table:

```
REVOKE UPDATE ON company_records.Employees FROM 'dept_manager'@'localhost';
```

Only the UPDATE privilege is revoked. SELECT still remains if previously granted.

(e) Implement SAVEPOINT:

```
START TRANSACTION;

INSERT INTO Employees (EmpId, EmpName, EmpSalary, DeptNo)
VALUES (501, 'rajat verma', 45000, NULL);

SAVEPOINT entry_alpha;

UPDATE Employees SET EmpSalary = 48000 WHERE EmpId = 501;

COMMIT;
```

SAVEPOINT entry_alpha marks a point after inserting the record. The subsequent UPDATE changes the salary to 48000. COMMIT makes both operations permanent. Rolling back to entry_alpha would undo the UPDATE but keep the INSERT.

EXPERIMENT 10

SQL Joins and Subqueries – DEPARTMENTS & EMPLOYEES

Schema:

Departments (DeptNo, DeptName, DeptLocation)

Employees (EmpId, EmpName, EmpSalary, DeptNo)

Table Creation & Data:

```
CREATE DATABASE IF NOT EXISTS company_joins;
USE company_joins;

CREATE TABLE IF NOT EXISTS Departments (
    DeptNo INT PRIMARY KEY,
    DeptName VARCHAR(50),
    DeptLocation VARCHAR(50)
);

CREATE TABLE IF NOT EXISTS Employees (
    EmpId INT PRIMARY KEY,
    EmpName VARCHAR(50),
    EmpSalary INT,
    DeptNo INT
);

INSERT INTO Departments VALUES
(10,'mathematics','block a'),(20,'commerce','block b'),
(30,'science','block c'),(40,'humanities','block d');

INSERT INTO Employees VALUES
(701,'amit kumar',45000,10),(702,'sunita devi',55000,20),
(703,'rahul sharma',38000,10),(704,'priya nair',60000,30),
(705,'vicky singh',42000,NULL);
```

(a) Inner Join – Employees with matching Department:

```
SELECT E.*, D.DeptName, D.DeptLocation
FROM Employees E
JOIN Departments D ON E.DeptNo = D.DeptNo;
```

Output (4 rows — vicky singh excluded as DeptNo is NULL):

701 | amit kumar | 45000 | 10 | mathematics | block a

702 | sunita devi | 55000 | 20 | commerce | block b

703 | rahul sharma | 38000 | 10 | mathematics | block a

704 | priya nair | 60000 | 30 | science | block c

(b) Left Outer Join – All employees with Department (if any):

```
SELECT E.EmpName, D.DeptName
FROM Employees E
LEFT OUTER JOIN Departments D ON E.DeptNo = D.DeptNo;
```

Output (5 rows — vicky singh shows NULL for DeptName):

amit kumar | mathematics

sunita devi | commerce

rahul sharma | mathematics

priya nair | science

vicky singh | NULL

(c) Right Outer Join – All Departments with Employee (if any):

```
SELECT E.EmpName, D.DeptName
FROM Employees E
RIGHT OUTER JOIN Departments D ON E.DeptNo = D.DeptNo;
```

Output (5 rows — humanities shows NULL for EmpName):

rahul sharma | mathematics

amit kumar | mathematics

sunita devi | commerce

priya nair | science

NULL | humanities

(d) Employees with salary > average salary (subquery):

```
SELECT * FROM Employees
WHERE EmpSalary > (SELECT AVG(EmpSalary) FROM Employees);
```

Average Salary = (45000+55000+38000+60000+42000)/5 = 48000

Output (employees above average):

702 | sunita devi | 55000 | 20

704 | priya nair | 60000 | 30